影像處理實習

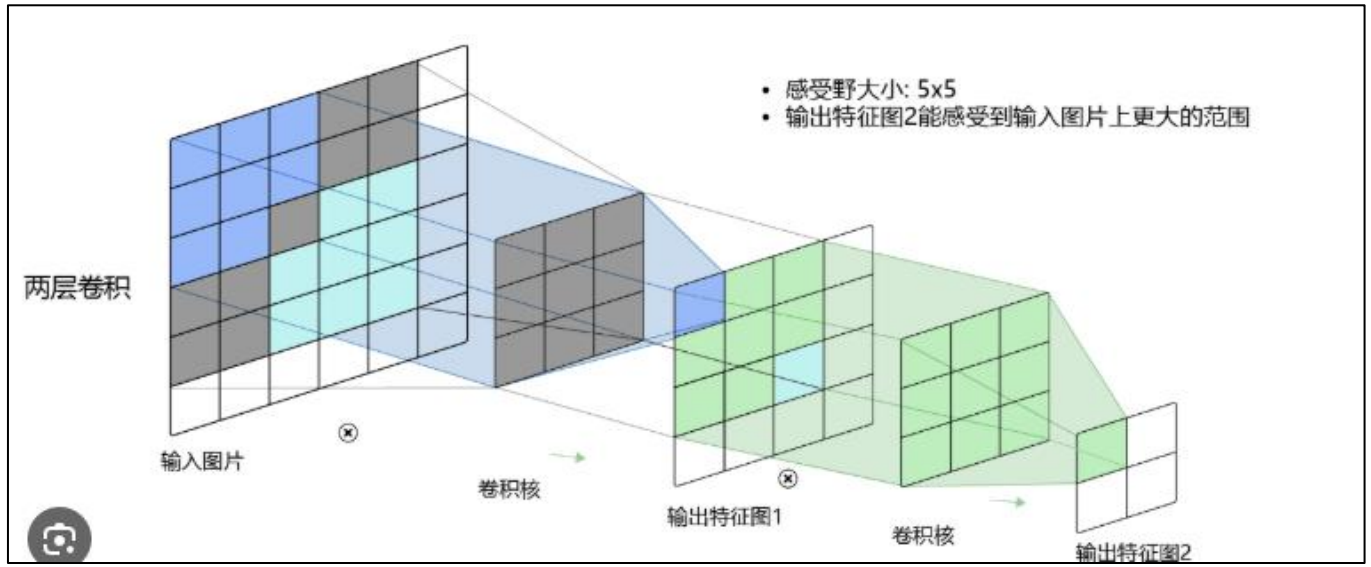
Week 5. 空間濾波

助教 陳立堯

影像捲積

Part 1. 捲積

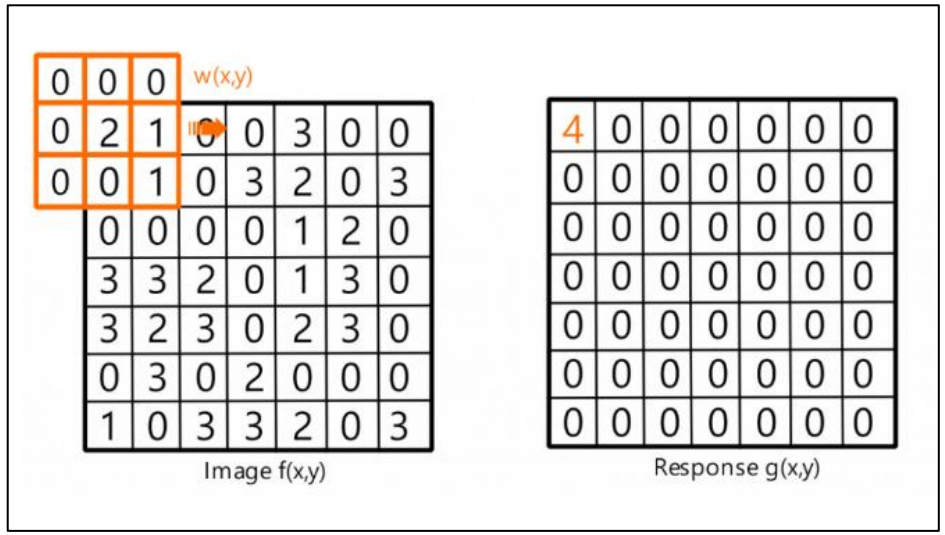
使用捲積核(kernel)，系統性地滑過整張影像，將捲積核的特性應用到整張影像後，輸出一張新的影像 (又可稱為特徵圖)，代表了我們希望捲積核想找到的特徵。



https://www.google.com/url?sa=i&url=https%3A%2F%2Fpaddlepedia.readthedocs.io%2Fen%2Flatest%2Ftutorials%2FCNN%2Fconvolution_operator%2FConvolution.html&psig=AOvVaw1yTtiar6e-NiB7YfRpQ9oE&ust=1760519914311000&source=images&cd=vfe&opi=89978449&ved=0CBUQjRxqFwoTCKje-qWuo5ADFQAAAAAdAAAAABAL

Part 1. Kernel

假設一個 3x3 kernel，W(x,y)是核中心 (又稱錨點Anchor Point)，kernel中的其他元素是相對於他而定的。核心初始對準(0, 0)，之後由左上至右下滑動，每次都與對應像素相乘。



https://ithelp.ithome.com.tw/articles/10322126

W(x-1,y-1)	W(x,y-1)	W(x+1,y-1)
W(x-1,y)	W(x,y)	W(x+1,y)
W(x-1,y+1)	W(x,y+1)	W(x+1,y+1)

Part 1. 捲積操作

```
import numpy as np
image = np.array([[1,2,3,4],
                  [5,6,7,8],
                  [9,10,11,12],
                  [13,14,15,16]])
kernel = np.array([[1,1,1],
                   [1,1,-1],
                   [1,1,1]])

output_image = np.zeros((2,2))

for y in range(output_image.shape[0]):
  for x in range(output_image.shape[1]):
    neighborhood = image[y:y+3, x:x+3]
    calculation = np.sum(neighborhood * kernel)
    output_image[y ,x] = calculation
print(output_image)
```

輸入
圖片

kernel

根據kernel
大小取框

像素值對應
kernel相乘

Part 1. Padding

功能：

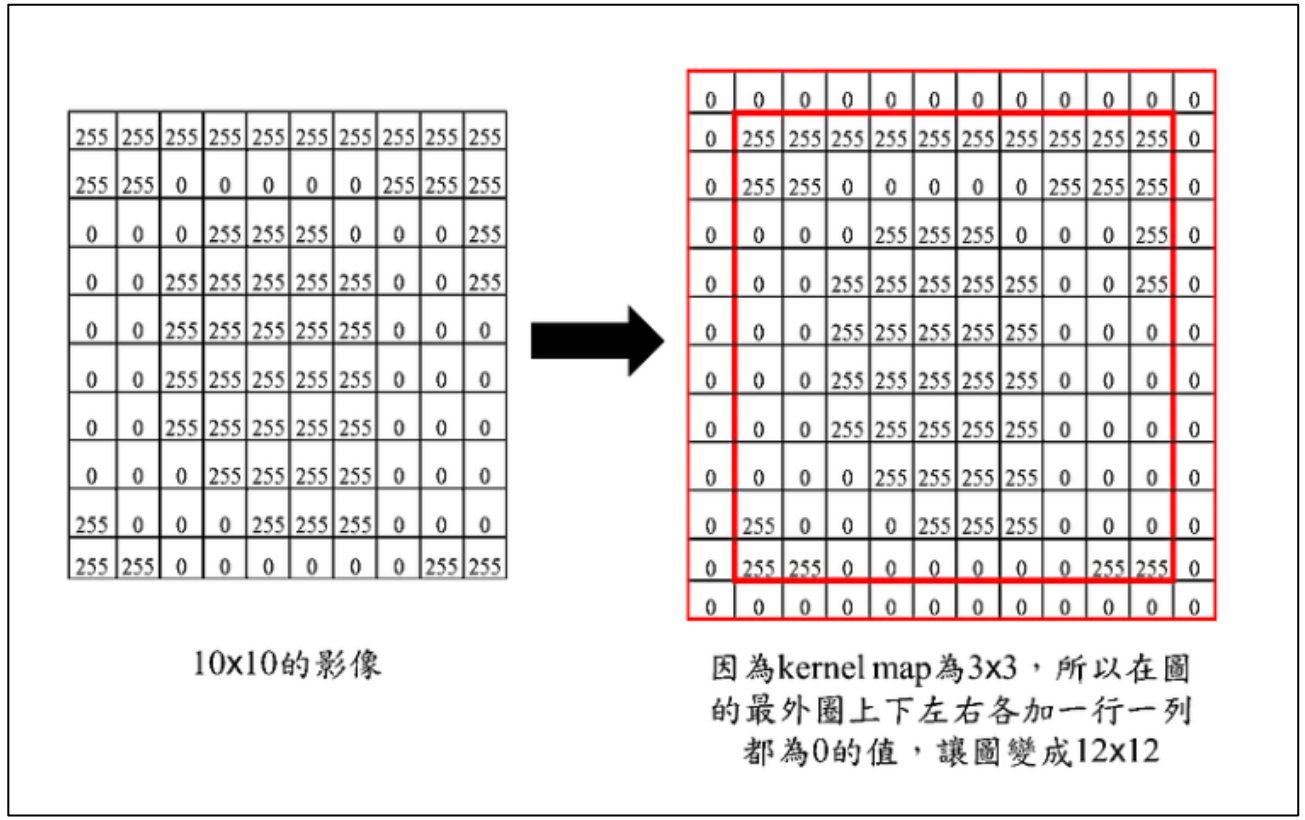
先讓原圖擴增尺寸，

之後能使捲積後的影像保持原本大小

常用的策略有 :

1. Valid padding (P=0)

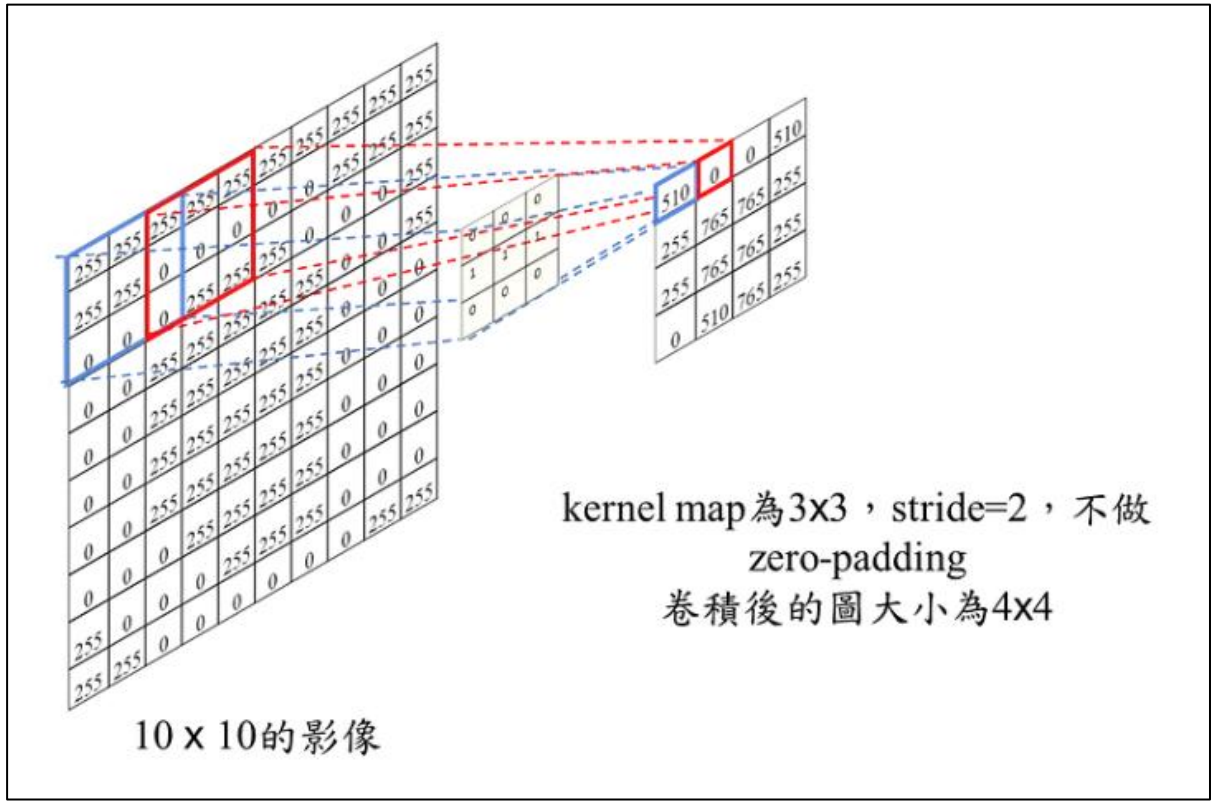
2. Same padding (P=(K-1) / 2)



Part 1. Stride

控制捲積核每次移動的步幅

可以有效減少計算量，讓輸出特徵變得更小。



https://chih-sheng-huang821.medium.com/%E5%8D%B7%E7%A9%8D%E7%A5%9E%E7%B6%93%E7%B6%B2%E8%B7%AF-convolutional-neural-network-cnn-%E5%8D%B7%E7%A9%8D%E8%A8%88%E7%AE%97%E4%B8%AD%E7%9A%84%E6%AD%A5%E4%BC%90-stride-%E5%92%8C%E5%A1%AB%E5%85%85-padding-94449e638e82

Part 1. 三者關係

W:原本長寬 | K:kernel size | S :步幅

New height/width = $((\boldsymbol{W} + \mathbf{2} * \boldsymbol{pad} - \boldsymbol{K}) / \boldsymbol{S}) + \mathbf{1}$

若希望padding大小相同，假設stride=1

New height = W = (W + 2*pad -K) / S + 1

$Pad = (K - 1) / 2$

本日作業

將7x7矩陣使用5x5捲積核捲積，輸出特徵矩陣要和原圖矩陣大小相同

請繳交輸出矩陣的截圖

檔案限制.jpg or .png

```
hw_image  =  np.array([
        [1,2,3,4,5,6,7],
        [2,4,5,5,6,7,7],
        [1,3,4,5,2,3,4],
        [5,6,3,2,4,5,6],
        [1,2,3,4,6,7,8],
        [2,7,6,8,3,4,5],
        [7,8,2,6,3,4,6]
])

kernel  =  np.array([
        [1,0,0,0,0],
        [0,1,0,0,0],
        [0,0,1,0,0],
        [0,0,0,1,0],
        [0,0,0,0,1]
])
```

7 x 7

5 x 5